29 @nestjs/config — ConfigModule / ConfigService, USES WITH EXAMPLES

theory/28-environment-variables.txt covers env vars in general and where raw process.env already lives in this project (main.ts's PORT, auth.guard.ts's VALID_API_KEY). This file is the next step: the actual package Nest projects use once process.env alone starts to hurt, with runnable examples against THIS project's real files.

NOT INSTALLED YET in this project — everything below is a designed example, same convention as theory/17's ApiKeyGuard.

```
npm install @nestjs/config
```

USE 1 — AUTO-LOADING .env, GLOBALLY, ONE LINE

Without this package you'd hand-write `import 'dotenv/config'` at the top of main.ts, before anything else runs. ConfigModule replaces that AND makes the values available for injection everywhere else, if you mark it global:

```
// app.module.ts
import { ConfigModule } from '@nestjs/config';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }), // reads .env once, at boot
    EmployeeModule, CategoryModule, UserModule, TaskModule,
  ],
  controllers: [AppController],
  providers: [AppService],
})
export class AppModule {}
```

`isGlobal: true` means you register ConfigModule ONCE, here, and every OTHER module (TaskModule, EmployeeModule, ...) can inject ConfigService without importing ConfigModule itself again — same DI convenience pattern as theory/07-dependency-injection.txt describes for providers.

```
// .env  (git-ignored)
PORT=4000
API_KEY=secret123
```

USE 2 — REPLACING THE HARDCODED VALID_API_KEY (auth.guard.ts)

This is the exact TODO already sitting in the codebase:

```
// auth.guard.ts, as it is today
const VALID_API_KEY = 'secret123'; // TODO: move to env/ConfigService in a real app
```

With ConfigService injected, the guard reads the key at request time instead of hardcoding it into the compiled source:

```ts
import { CanActivate, ExecutionContext, Injectable, UnauthorizedException } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { Request } from 'express';

@Injectable()
export class AuthGuard implements CanActivate {
  constructor(private readonly config: ConfigService) {} // DI'd, same as RolesGuard's Reflector

  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest<Request>();
    const key = request.headers['x-api-key'];
    const validKey = this.config.get<string>('API_KEY'); // reads process.env.API_KEY under the hood

    if (key !== validKey) {
      throw new UnauthorizedException('Invalid or missing API key');
    }
    return true;
  }
}
```

Nothing about auth.guard.ts's SHAPE changes — still a CanActivate, still throws UnauthorizedException the same way (theory/17-guards.txt, theory/24-exception-handling.txt). Only where the key comes from changes: a real secret from .env in prod, easily swapped in tests.

USE 3 — VALIDATING REQUIRED VARS AT STARTUP, NOT MID-REQUEST

Without validation, a missing API_KEY in production means `this.config.get('API_KEY')` silently returns undefined, and EVERY request fails with a confusing 401 — you find out from a user, not from the deploy. A Joi schema makes the app refuse to boot instead:

```ts
npm install joi

// app.module.ts
import * as Joi from 'joi';

ConfigModule.forRoot({
  isGlobal: true,
  validationSchema: Joi.object({
    PORT: Joi.number().default(3000),
    API_KEY: Joi.string().required(), // app crashes on startup if this is missing
  }),
})
```

Result: `npm run start` immediately throws a clear "API_KEY is required" error at boot if `.env` is misconfigured, instead of the failure surfacing later as mysterious 401s.

USE 4 — TYPED, INJECTED PORT (replacing process.env.PORT in main.ts)

main.ts currently reads process.env directly, which is fine in main.ts specifically (ConfigService isn't available until the Nest app instance exists) — but once the app is built, you can pull the SAME value back out through the app's own DI container instead of reaching for the global again:

```ts
// main.ts
import { ConfigService } from '@nestjs/config';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  const config = app.get(ConfigService); // pull ConfigService out of the app's DI container
  const port = config.get<number>('PORT') ?? 3000;

  app.enableShutdownHooks();
  await app.listen(port);
  console.log(`Server is running on port ${port}`);
}
```

`app.get(X)` (vs constructor injection) is how you reach into DI from OUTSIDE a class, which is exactly bootstrap()'s situation — it's a plain function, not a Nest-managed class.

USE 5 — NAMESPACED CONFIG WITH registerAs (GROUPING RELATED VALUES)

Flat keys (`'API_KEY'` , `'DB_HOST'` , `'DB_PORT'`) get messy fast. `registerAs` groups them into a typed namespace:

```ts
// config/database.config.ts
import { registerAs } from '@nestjs/config';

export default registerAs('database', () => ({
  host: process.env.DB_HOST,
  port: parseInt(process.env.DB_PORT ?? '5432', 10), // note: manual parseInt — env vars are always
strings
}));

// app.module.ts
ConfigModule.forRoot({ isGlobal: true, load: [databaseConfig] })

// anywhere else
this.config.get<string>('database.host');
this.config.get<number>('database.port');
```

Useful the moment a project has more than a couple of related settings (this project doesn't have a real database module yet, but this is exactly where DatabaseService's future real connection config would live — see theory/27-lifecycle-events.txt's DatabaseService example).

USE 6 — MOCKABLE IN TESTS (THE DI PAYOFF)

This is what raw process.env can't do cleanly. Testing AuthGuard with ConfigService means you can fake the config WITHOUT touching real env vars or global state:

```ts
const guard = new AuthGuard({
  get: () => 'secret123',
} as unknown as ConfigService); // fake ConfigService, no real .env needed
```

Or, in a Nest TestingModule:

```ts
const module = await Test.createTestingModule({
  providers: [
```

```
    AuthGuard,
    { provide: ConfigService, useValue: { get: () => 'secret123' } },
  ],
}).compile();
```

Every test controls its own "API_KEY" independently — no shared process.env mutation between test files, no test-ordering bugs.

★ RULE OF THUMB

Reach for ConfigModule/ConfigService the moment ANY of these are true: you have more than ~2-3 env vars, you want the app to REFUSE to start on missing config rather than fail confusingly at request time (validationSchema), or you're writing unit tests around code that reads config (ConfigService mocks cleanly, process.env doesn't). Below that bar, main.ts's current `process.env.PORT ?? 3000` is genuinely fine — don't add the dependency just because it exists.

See also: theory/28-environment-variables.txt, theory/17-guards.txt, theory/07-dependency-injection.txt, theory/27-lifecycle-events.txt